Take a peek inside that error message. RSpec shows both the message name and arguments we sent to our double; this is already more information than a typical Ruby NoMethodError.

RSpec describes the object only as #<Double (anonymous)>, with no hint as to what we're using it for. You can get a little more detail in the error message by naming the role the double plays—just pass a name to the double method.

Since this object is standing in for a Ledger instance, let's name it 'Ledger':

```
>> ledger = double('Ledger')
=> #<Double "Ledger">
>> ledger.record(an: :expense)
RSpec::Mocks::MockExpectationError: #<Double "Ledger"> received unexpected ↩
message :record with ({:an=>:expense})
    « backtrace truncated »
```

The error message contains the role name now. This extra information comes in handy when you're using multiple doubles in the same example and need to tell them apart.

This same double method can create any of the other kinds of test doubles you'll use in your specs: stubs, mocks, spies, and null objects. Over the next few sections, we're going to take a look at each of these in turn.

Stubs

As we said at the start of this chapter, stubs are simple. They return pre-programmed, canned responses. Stubs are best for when you're simulating *query methods*—that is, methods that return a value but don't perform side effects.

The simplest way to define a stub is to pass a hash of method names and return values to the double method:

```
>> http_response = double('HTTPResponse', status: 200, body: 'OK')
=> #<Double "HTTPResponse">
>> http_response.status
=> 200
>> http_response.body
=> "OK"
```

As an alternative, you can perform these two steps (creating the stub and setting up the canned messages) separately. To do so, pass that same hash of method names and return values into allow(...).to receive_messages(...), like so:

```
>> http_response = double('HTTPResponse')
=> #<Double "HTTPResponse">
>> allow(http_response).to receive_messages(status: 200, body: 'OK')
```